Autoorganización

Aprendizaje competitivo simple

En los modelos de aprendizaje competitivo:

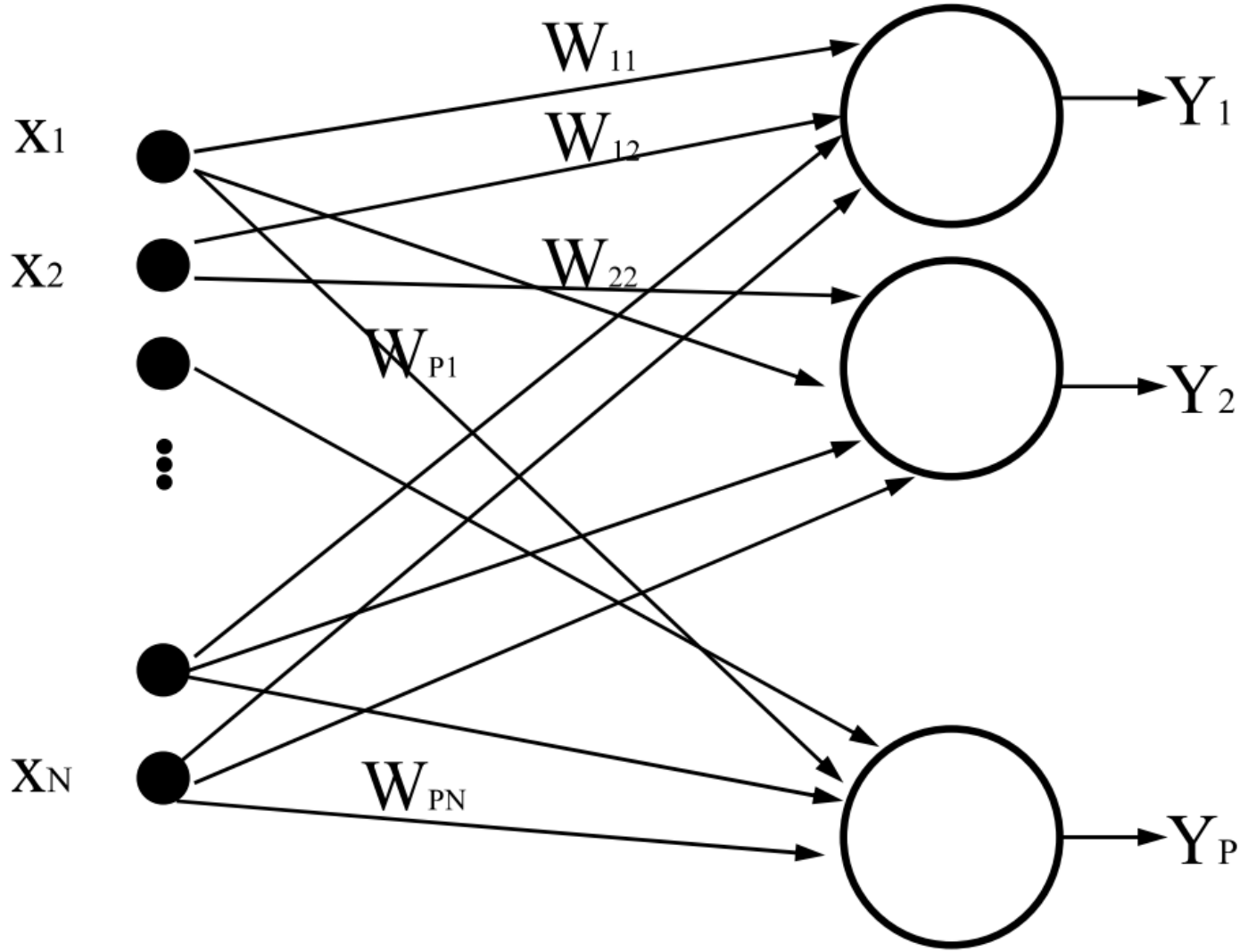
- solo una neurona se activa cada vez que se presenta una entrada.
- las unidades de salida compiten entre sí. La ganadora se lo lleva todo
- se forman grupos o categoría en los datos de entrada
- el número de grupos o categorías coincide con el número de unidades de salida

Modelo de aprendizaje Competitivo Simple (Arquitectura)

N neuronas de entrada

P neuronas de salida

NxP pesos

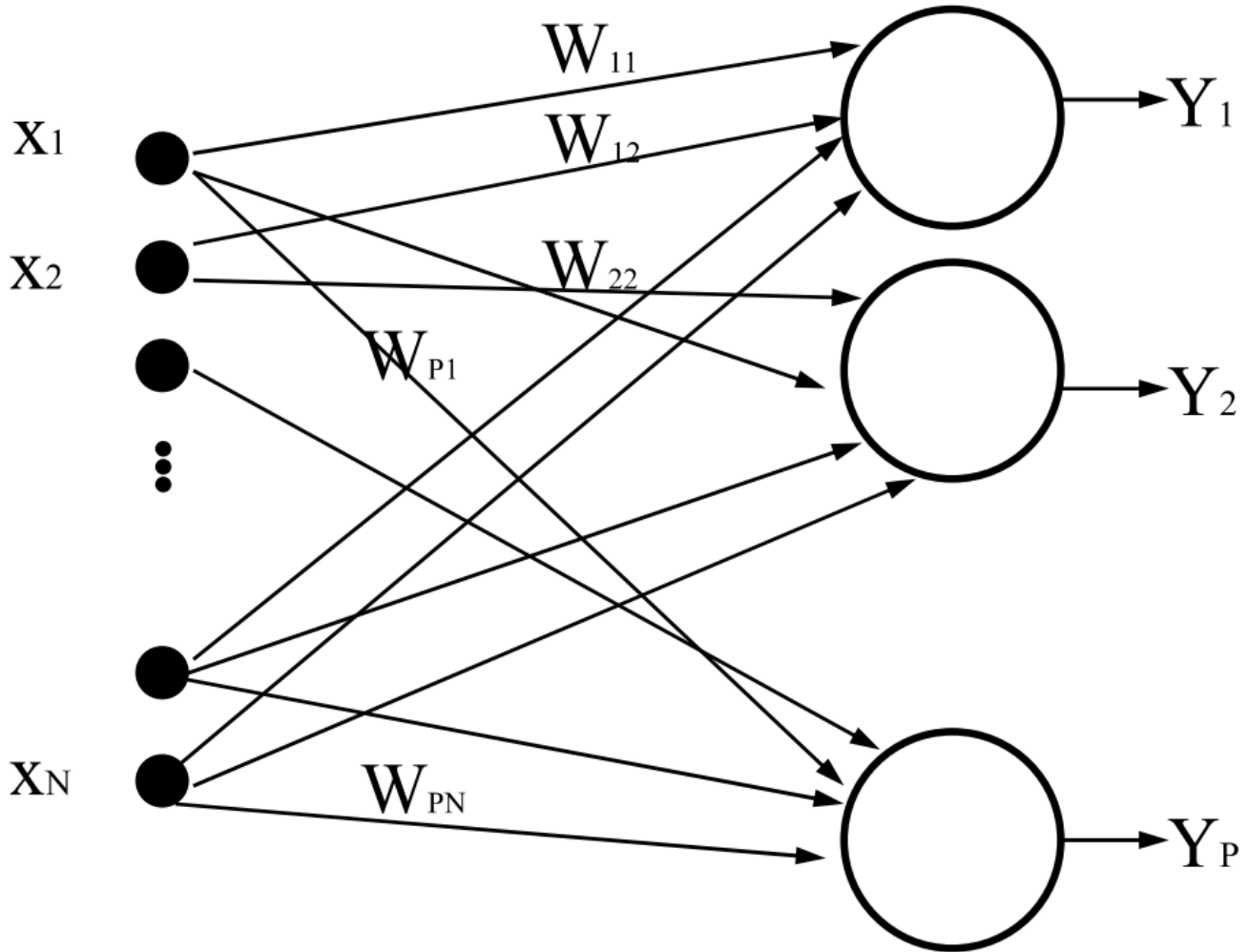


Modelo de aprendizaje Competitivo Simple (Dinámica de la Computación)

$$h_i = \sum_{j=1}^{N} W_{ij} X_j$$

$$i = 1, 2 \ldots P$$

$Y_i \in \{0,1\}$,i=1,…,P



Modelo de aprendizaje Competitivo Simple (Dinámica de la Computación)

- Los pesos están normalizados
- Las salidas son binarias
- Solo una neurona se activa en respuesta a un estímulo de entrada

$$h_i = \sum_j W_{ij} X_j = \vec{W}_i \vec{X}$$

$$i^* = \arg\max(h_i)$$

Modelo de aprendizaje Competitivo Simple (Aprendizaje)

Se activa la unidad de salida con mayor valor de su potencial sináptico o nivel de activación h_i,
sea la unidad ganadora i^*

- Geométricamente, $\overrightarrow{W_i}$es el vector de pesos más cercano al vector de entrada $\vec{X}$
- Los pesos de la unida ganadora se actualizan para que queden más próximos al vector de entrada actual

Aprendizaje Competitivo Simple

Entendí

Teorema

Si i^* es la unidad de proceso ganadora cuando de introduce el patrón de entrada $\vec{X}$, entonces

$$\mathrm{d}(\vec{X},\vec{W}_{i^*}) \leq \mathrm{d}(\vec{X},\vec{W}_i),\ i=1,2,\ldots,p$$

Demostración:

$$(\mathrm{d}(\vec{X},\vec{W}_{i^*}))^2 = \left\|\vec{X}-\vec{W}_{i^*}\right\|^2 = (\vec{X}-\vec{W}_{i^*})^t(\vec{X}-\vec{W}_{i^*}) = \vec{X}^t\vec{X} - 2\vec{W}_{i^*}{}^t\vec{X} +$$

$$\underbrace{\vec{W}_{i^*}{}^t\vec{W}_{i^*}}_{=1} = \vec{X}^t\vec{X} \underbrace{-2\vec{W}_{i^*}{}^t\vec{X}}_{=-2h_{i^*}} + \underbrace{\vec{W}_i{}^t\vec{W}_i}_{=1} \leq \vec{X}^t\vec{X} \underbrace{-2\vec{W}_i{}^t\vec{X}}_{=-2h_i} + \vec{W}_i{}^t\vec{W}_i$$

$$= \left\|\vec{X}-\vec{W}_i\right\|^2 = \mathrm{d}(\vec{X},\vec{W}_i)^2$$

Aprendizaje Competitivo Simple II

- Potencial sináptico:

$$h_i = w_{i1}x_1 + w_{i2}x_2 + \ldots + w_{iN}x_N - \theta_i$$

$$\theta_i = \frac{1}{2}\left(w_{i1}^2 + w_{i2}^2 + \cdots + w_{iN}^2\right)$$

- Dinámica de la computación

Para cada patrón **sólo hay una neurona ganadora**

$$\mathrm{Y}_i = \begin{cases} 1 & \text{si } h_i = \max\{h_1, h_2, \ldots, h_\mathrm{P}\} \\ 0 & \end{cases}$$

i=1,2,…,P

Modelo de aprendizaje Competitivo Simple (Aprendizaje)

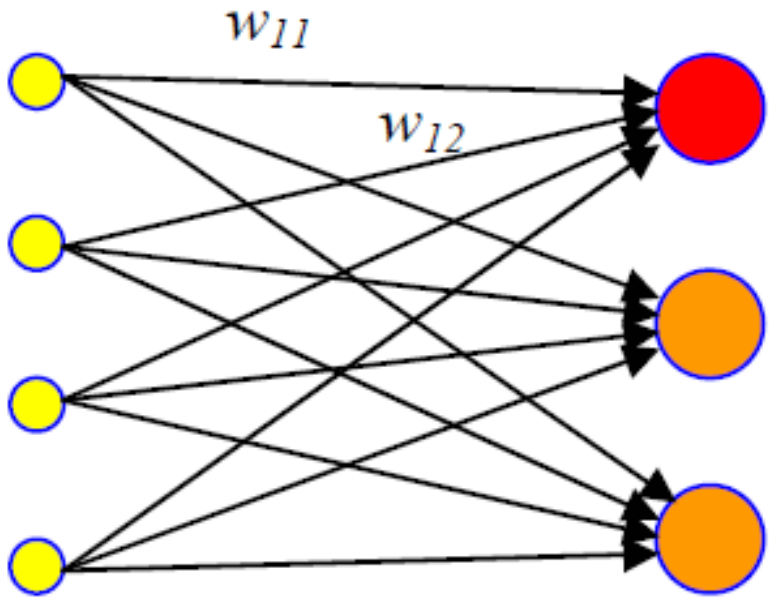
Se empieza con un conjunto de pesos aleatorios pequeños. En cada paso:

1. Un vector de entrada nuevo se presenta a la red
2. Se calculan los niveles de activación de las neuronas para encontrar la ganadora i^*
3. $\overrightarrow{W}_{i^*}$, el vector de pesos de la unidad ganadora se actualiza

Aprendizaje Competitivo Simple (Ley de aprendizaje)

Sea i^* la unidad ganadora y μ>0 una constante llamada tasa de aprendizaje

$$\overrightarrow{W}_i^*(\mathrm{t}+1) = \overrightarrow{W}_i^*(t) + \mu(\vec{X} - \overrightarrow{W}_i^*(t))$$

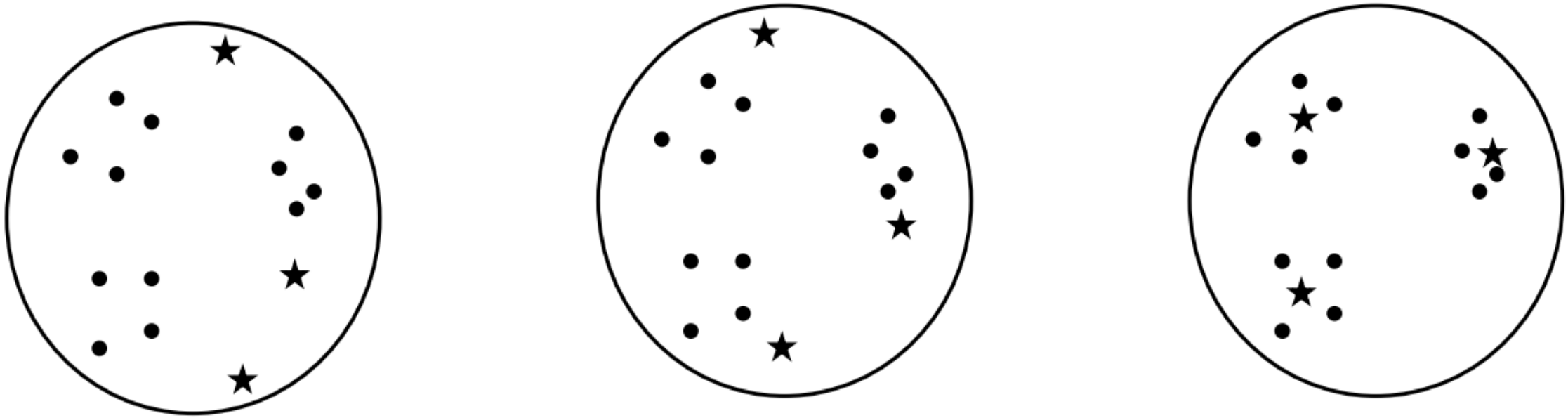


o bien

$W_{ij}^*(t+1) = W_{ij}^*(t) + \mu\Delta W_{ij}^*(t)$, donde

$\Delta W_{ij}^*(t) = X_j(t) - W_{ij}^*(t)$

Aprendizaje Competitivo Simple (Resultado del aprendizaje)



Aprendizaje Competitivo Simple (Resultado del aprendizaje)

https://www.youtube.com/watch?v=xQ1sYhIU9dk

Regla de aprendizaje competitivo simple (Deducción)

Se trata de encontrar P vectores de pesos sinápticos

$$\overrightarrow{W}_1, \overrightarrow{W}_2, \ldots, \overrightarrow{W}_p,$$

tales que la función suma de errores cuadráticos sea mínima, es decir, se trata de minimizar la expresión

$$E = \sum_{i=1}^{P} \sum_{j=1}^{K} a_{ij} \left\| \vec{x}(j) - \overrightarrow{W}_i \right\|^2$$

donde $a_{ij} = \begin{cases} 1 \text{ si } \vec{x}(j) \in C_i \\ 0 \text{ en otro caso} \end{cases}$

a_{ij} indica si el patrón de entrada $\vec{x}(j)$ es, o no, de la clase C_i, pero dicha función **no es conocida**

Regla de aprendizaje competitivo simple individualizado (Deducción)

- Sea $\vec{x}(k)$el patrón que introducimos en la red en la iteración k.
- Se quiere modificar los vectores de pesos sinápticos de modo que se minimice la expresión del error que depende de dicho patrón:

$$E = \sum_{i=1}^{p} a_{ik} \left\| \vec{x}(k) - \overrightarrow{W_i}(k) \right\|^2$$

- Se usará la técnica del gradiente descendente

$$\Delta \overrightarrow{W_i}(k) = -\lambda \frac{\partial E}{\partial \overrightarrow{W_i}(k)} = -2\, a_{ik}(\vec{x}(k) - \overrightarrow{W_i}(k))$$

Regla de aprendizaje competitivo simple individualizado (Deducción)

el patrón $\vec{x}(k)$ sólo puede pertenecer a una de las P clases, vamos a estimar los valores desconocidos de $a_{1k}, \cdots, a_{ik}, \cdots, a_{Pk}$.

$$\hat{a}_{rk} = \begin{cases} 1 & \text{si } \left\|\vec{x}(k) - \overrightarrow{W}_r(k)\right\| < \left\|\vec{x}(k) - \overrightarrow{W}_i(k)\right\|_{,\,\forall\, i \neq r} \\ 0 & \text{en otro caso} \end{cases}$$

Regla de aprendizaje competitivo simple individualizado

$$\overrightarrow{W_i}(\mathrm{k}+1)=\overrightarrow{W_i}(k)+\Delta\overrightarrow{W_i}(k)$$

Donde

$$\Delta\overrightarrow{W_i}(k)=\begin{cases}\eta(k)\left(\vec{x}(k)-\overrightarrow{W_i}(k)\right) & \text{si } i=r\\ 0 & \text{si } \mathrm{i}\neq r\end{cases}$$

r es la unidad ganadora según el teorema anterior (transparencia7)

Regla de aprendizaje competitivo simple individualizado (Interpretación)

Si r es la unida ganadora

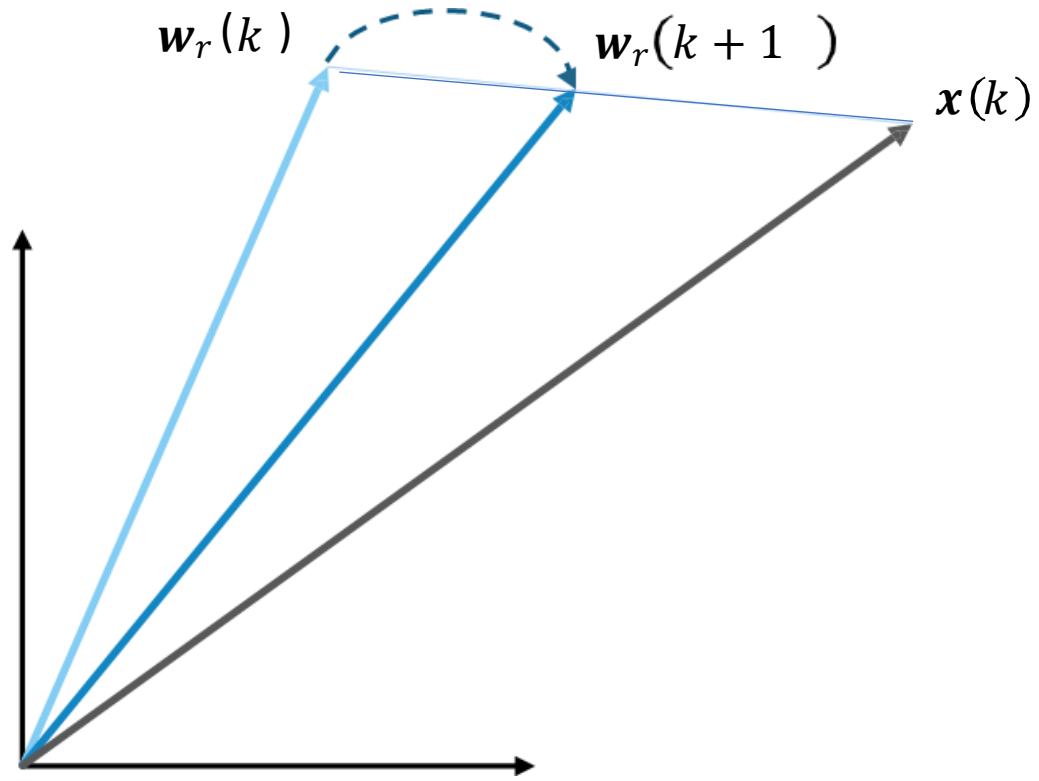
$$\overrightarrow{W_r}(k+1) = \overrightarrow{W_r}(k) + \eta(k)\left(\vec{x}(k) - \overrightarrow{W_r}(k)\right)$$

es decir

$$\overrightarrow{W_r}(k+1) = (1-\eta(k))\overrightarrow{W_r}(k) + \eta(k)\vec{x}(k)$$

El nuevo vector de pesos es una combinación lineal de $\vec{x}(k)$ y $\overrightarrow{W_r}(k)$

Ley de aprendizaje competitivo individualizado
Interpretación geométrica



Regla de aprendizaje competitivo por lotes (Deducción)

- Se actualizan los vectores de pesos sinápticos después de introducir todos los patrones de entrada entonces se trata de minimizar la expresión

$$E = \sum_{i=1}^{P} \sum_{j=1}^{K} a_{ij} \left\| \vec{x}(j) - \overrightarrow{W_i} \right\|^2$$

donde $a_{ij} = \begin{cases} 1 \text{ si } \vec{x}(j) \in C_i \\ 0 \text{ en otro caso} \end{cases}$

a_{ij} indica si el patrón de entrada $\vec{x}(j)$ es, o no, de la clase C_i, pero dicha función **no es conocida**

Regla de aprendizaje competitivo por lotes (Deducción)

- Se usará la técnica del gradiente descendente

$$\Delta \vec{W}_i(k) = -\lambda \frac{\partial E}{\partial \vec{W}_i} = -2\sum_{j=1}^{K} a_{ij}(\vec{x}(j) - \vec{W}_i(k))$$

- La regla de aprendizaje es la siguiente

$$\vec{W}_i(\mathrm{k}+1) = \vec{W}_i(k) + \Delta\vec{W}_i(k)$$

donde $\Delta \vec{W}_i(k) = \mu(k) \sum_{j=1}^{K} a_{ij}(\vec{x}(j) - \vec{W}_i(k))$

a_{ij} se estiman como en la transparencia 14

ALGORITMO DE APRENDIZAJE COMPETITIVO INDIVIDUALIZADO

- Paso 0: Inicializar pesos sinápticos, por ejemplo, usar P patrones de entrenamiento. K=1.
- Paso 1: Elegir aleatoriamente un patrón de entrenamiento.
- Paso 2: Calcular los potenciales sinápticos
- Paso 3: Determinar la neurona ganadora r, es decir, la de mayor potencial sináptico

$$h_{i^*} = \max\{h_1, h_2, \dots, h_P\}$$

- Paso 4: Actualizar el peso de la neurona r ganadora:

$$\overrightarrow{W}_i^*(t+1) = \overrightarrow{W}_i^*(t) + \mu(\vec{X} - \overrightarrow{W}_i^*(t))$$

Hasta que se alcance el criterio de parada elegido

ALGORITMO DE APRENDIZAJE COMPETITIVO POR LOTES

- Paso 0: Inicializar pesos sinápticos, por ejemplo, usar P patrones de entrenamiento. K=1.
- Paso 1: Elegir aleatoriamente M patrones de entrada.
- Paso 2: Calcular los potenciales sinápticos $h_1(k), h_2(k), \ldots, h_P(k)$ para cada patrón de entrada $\vec{X}(k)$, k=1,2,…,M.
- Paso 3: Determinar la neurona ganadora i^*, es decir, la de mayor potencial sináptico

$$h_{i^*}(k) = max\{h_1(k), h_2(k), \ldots, h_P(k)\}$$

Poner $\hat{a}_{ij} = \begin{cases} 1 \text{ si } i \text{ es la unidad ganadora para } \overrightarrow{X}(j), \\ 0 \quad en\ otro\ caso \end{cases}$

- Paso 4: Actualizar el peso de cada neurona i como:

$$\overrightarrow{W_i}(t+1) = \overrightarrow{W_i}(t) + \mu \sum_{k=1}^{M} \hat{a}_{ij}(\vec{X}(k) - \overrightarrow{W_i}(t))$$

Características del aprendizaje competitivo :

La configuración final depende de la inicialización:

- Pesos iniciales
- Orden de presentación de los patrones de entrada
- Tasa de aprendizaje

Características del aprendizaje competitivo

- Una neurona ganadora tiene más oportunidades de ganar la próxima vez que una entrada es vista
- El aprendizaje podría resultar inestable

Esto se podría resolver tomando la tasa de aprendizaje decreciente conforme se avance en las iteraciones

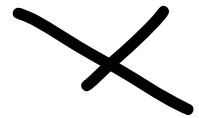
$$\eta\,(k) = \eta_0 \left(1 - \frac{k}{T}\right)$$

$\eta_0 \in (0,1]$ tasa de aprendizaje inicial

T es el número máximo de iteraciones de entrenamiento

Se puede elegir otra función decreciente para la tasa de aprendizaje

Características del aprendizaje competitivo

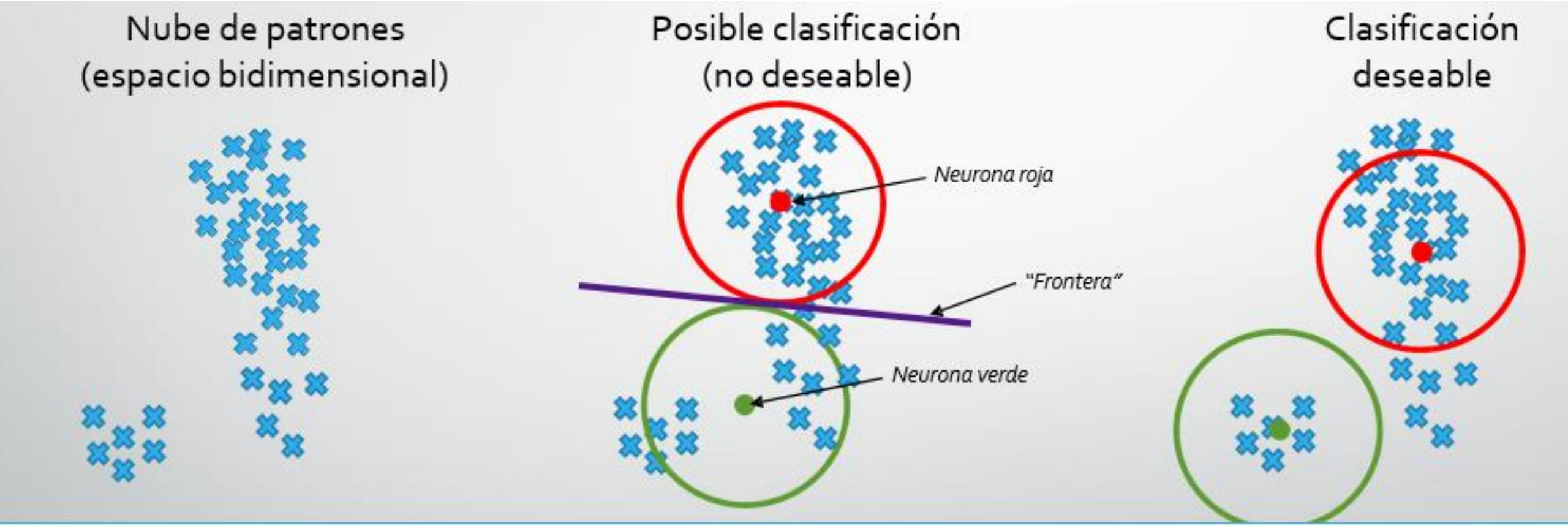


- Algunas neuronas de salida puede que nunca se activen (neuronas muertas)

Esto puede compensarse de diferentes formas

- ✓ eligiendo los vectores sinápticos iniciales de las unidades de proceso iguales a patrones del conjunto de entrenamiento, en lugar de elegir valores aleatorios
- ✓ restando un valor al potencial sináptico de las unidades que ganan con más frecuencia e ir incrementando proporcionalmente al número de veces que ganen (mecanismo de conciencia).
- ✓ actualizando las unidades no ganadoras pero en una cantidad más pequeña (autoorganización)

Gran diferencia entre el tamaño de los grupos



El aprendizaje podría definirse así

$$\Delta \mathbf{w}_i(k) = \begin{cases} \eta(k)\dfrac{1}{\hat{n}_i}(\mathbf{x}(k) - \mathbf{w}_i(k)) & \text{si } i = r \\ 0 & \text{si } i \neq r \end{cases}$$

Gran diferencia entre el tamaño de los grupos

Debe elegirse como criterio minimizar

$$E = \sum_{i=1}^{P} \frac{1}{n_i} \sum_{j=1}^{K} a_{ij} \left\| \vec{x}(j) - \overrightarrow{W_i} \right\|^2$$

Siendo n_i el número de patrones del grupo i, es decir $n_i = \sum_{j=1}^{K} a_{ij}$.

En este caso, el **aprendizaje por lotes** viene dado por

$$\overrightarrow{W_i}(\mathrm{t}+1) = \overrightarrow{W_i}(t) + \mu(t) \frac{1}{\hat{n}_i} \sum_{k=1}^{M} \hat{a}_{ij} (\vec{X}(\mathrm{k}) - \overrightarrow{W_i}(t))$$

donde $\hat{n}_i = \sum_{j=1}^{K} \hat{a}_{ij}$, para la unidad ganadora.

$$\overrightarrow{W_i}(\mathrm{t}+1) = (1 - \mu(t)) \overrightarrow{W_i}(t) + \mu(t) \frac{1}{\hat{n}_i} \sum_{k=1}^{M} \hat{a}_{ij} \vec{X}(\mathrm{k})$$

el nuevo valor del peso sináptico es una combinación lineal de su valor actual con el patrón promedio de los patrones asignados a la clase i.

Gran diferencia entre el tamaño de los grupos

Para el **aprendizaje individualizado** viene dado por

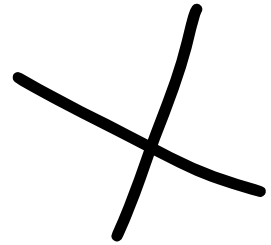
$$\overrightarrow{W_i}(\mathrm{t}+1) = \overrightarrow{W_i}(t) + \mu(t)\frac{1}{\hat{n}_i}(\vec{X}(\mathrm{k})\text{-}\overrightarrow{W_i}(t))$$

donde $\hat{n}_i = \sum_{j=1}^{K} \hat{a}_{ij}$, para la unidad ganadora.

$$\overrightarrow{W_i}(\mathrm{t}+1) = (1-\mu(t)\frac{1}{\hat{n}_i})\overrightarrow{W_i}(t) + \mu(t)\frac{1}{\hat{n}_i}\vec{X}(\mathrm{k})$$

el nuevo valor del vector sináptico es una combinación lineal de su valor actual con el patrón de entrada, ponderado de forma inversa con el tamaño de la clase a la que pertenece (se modifica menos el vector sináptico cuantos más patrones tenga la clase *i* a la que ha sido asignado el patrón de entrada.

Ejemplo



Tenemos una serie de patrones bidimensionales

$\vec{X}1=(4,2)^t$, $\vec{X}2=(1,3)^t$, $\vec{X}3=(1,1)^t$, $\vec{X}4=(4,4)^t$, $\vec{X}5=(2,0)^t$

que queremos agrupar en tres clases. Los valores iniciales para los pesos sinápticos de las tres neuronas son los siguientes

$\overrightarrow{W_1} = (1,2)^t, \overrightarrow{W_2} = (2,1)^t, \overrightarrow{W_3} = (3,3)^t$

El primer patrón de entrada es $\vec{X}=(4,2)^t$, calculamos los potenciales sinápticos

$h_1(1)=4*1+2*2-0.5(1+4)=5.5$

$h_2(1)=4*2+2*1-0.5(4+1)=7.5$

$h_3(1)=4*3+2*3-0.5(9+9)=9$

Ejemplo

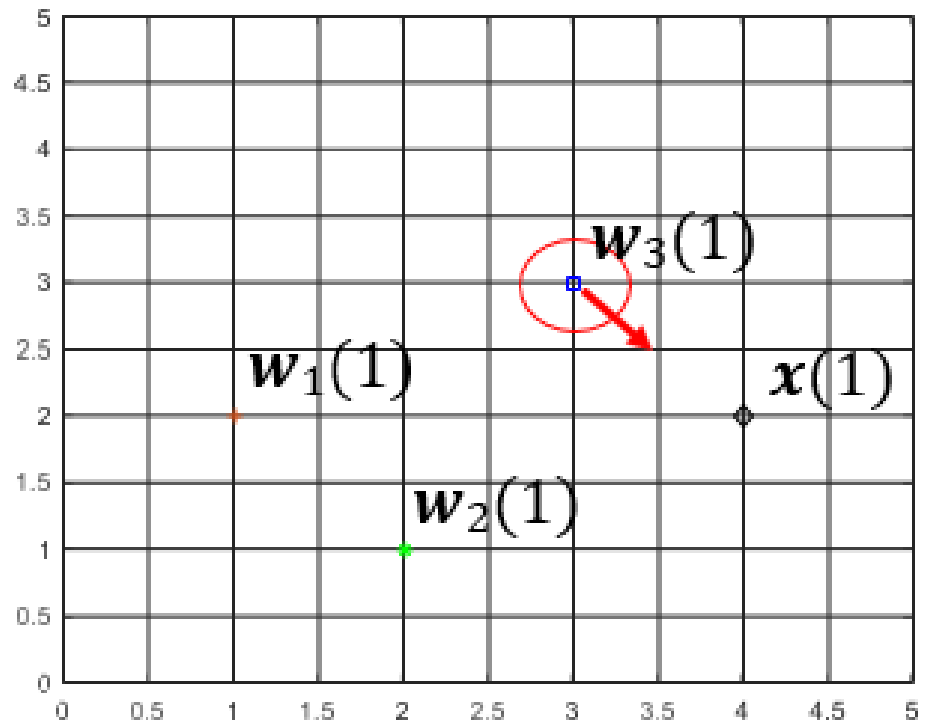
Actualiza el peso sináptico de la neurona ganadora y también la tasa de aprendizaje para la siguiente iteración. Utiliza $\eta(1) = 0.5$

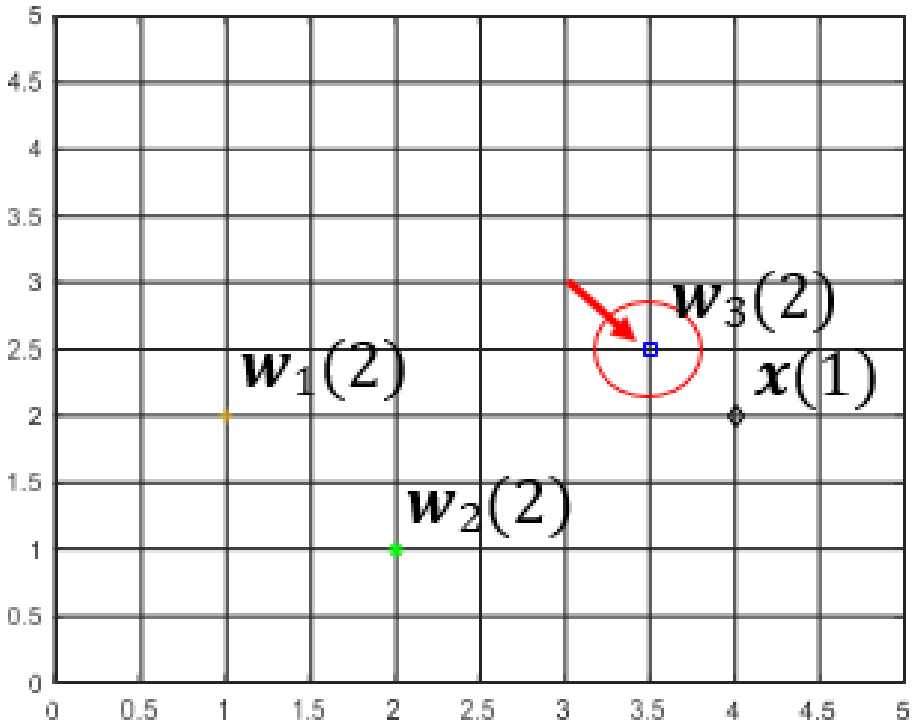
$$W_{31}(2) = 3 + 0.5(4 - 3) = 3.5$$
$$W_{32}(2) = 3 + 0.5(2 - 3) = 2.5$$

Entonces los nuevos pesos son

$\overrightarrow{W_1}(2) = (1,2)^t, \overrightarrow{W_2}(2) = (2,1)^t, \overrightarrow{W_3}(2) = (3.5,2.5)^t$

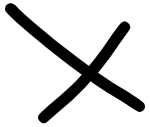
Primera iteración





Seguir iterando…..

Ejercicio



Una plataforma de streaming desea identificar qué usuarios tienen mayor probabilidad de suscribirse a su plan **Premium** y cuáles permanecerán en el plan **Gratis**. Para optimizar su estrategia de marketing, la empresa necesita clasificar a los usuarios en tres categorías:

1. **Usuarios Premium Seguros (PS)**: Ya están suscritos o tienen alta probabilidad de pagar por la suscripción.
2. **Usuarios Indecisos (IN)**: Usan la plataforma frecuentemente pero aún no han decidido pagar.
3. **Usuarios Gratis Seguros (GS)**: Usan la plataforma pero muestran poco interés en pagar por el servicio.

Ejercicio

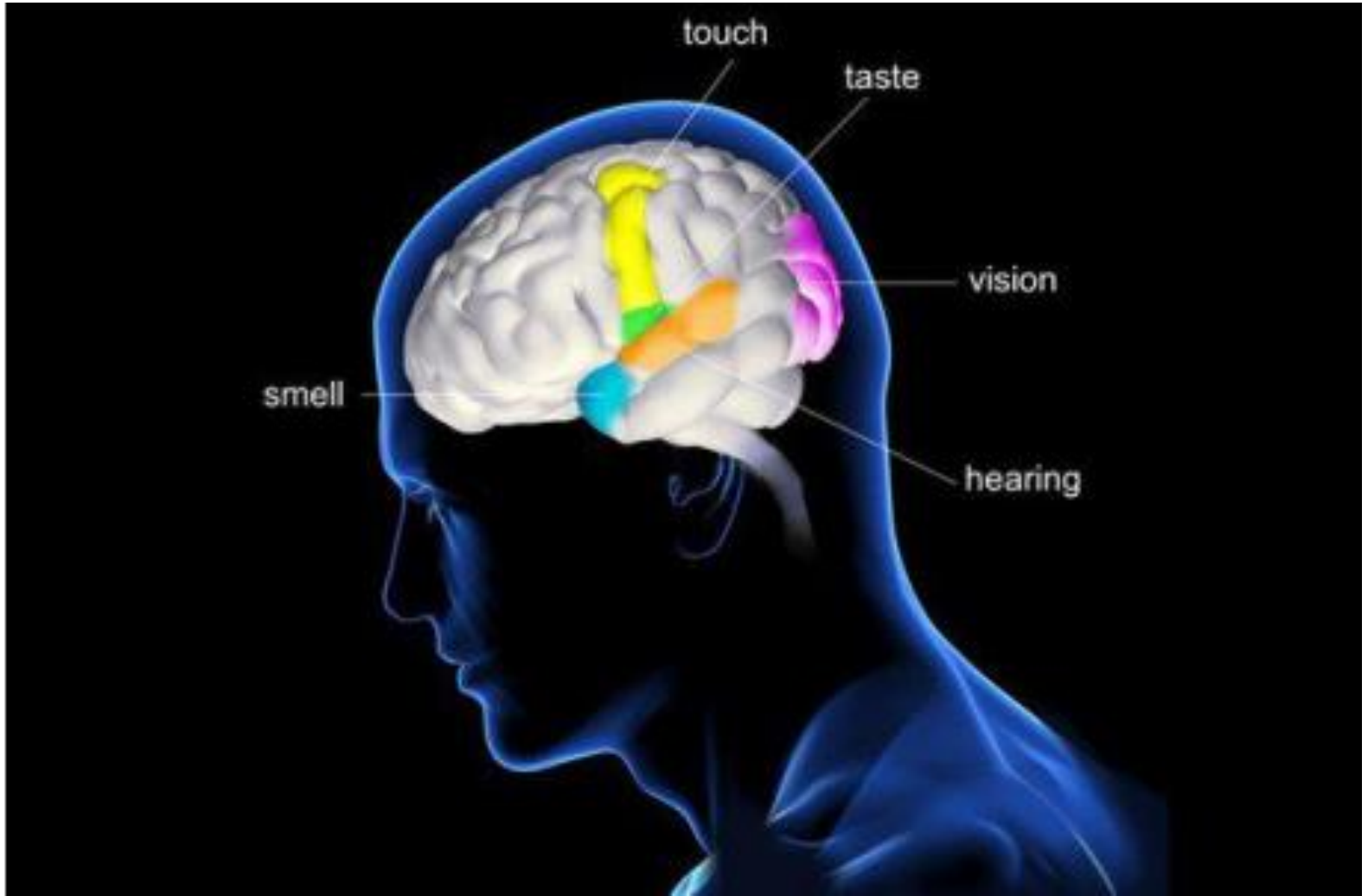
Se dispone de una muestra de 500 usuarios donde cada usuario se describe con un vector de características:

- Tiempo promedio de uso diario.
- Cantidad de anuncios vistos sin cerrar la app.
- Número de veces que exploran la sección premium.
- Historial de prueba de versiones premium.
- Dispositivo utilizado (móvil, PC, Smart TV).

Diseña una red una **red neuronal competitiva** para la desarrollar tarea descrita

Autoorganización
Motivación biológica

En el cerebro, las entradas sensoriales se representan por mapas ordenados topológicamente

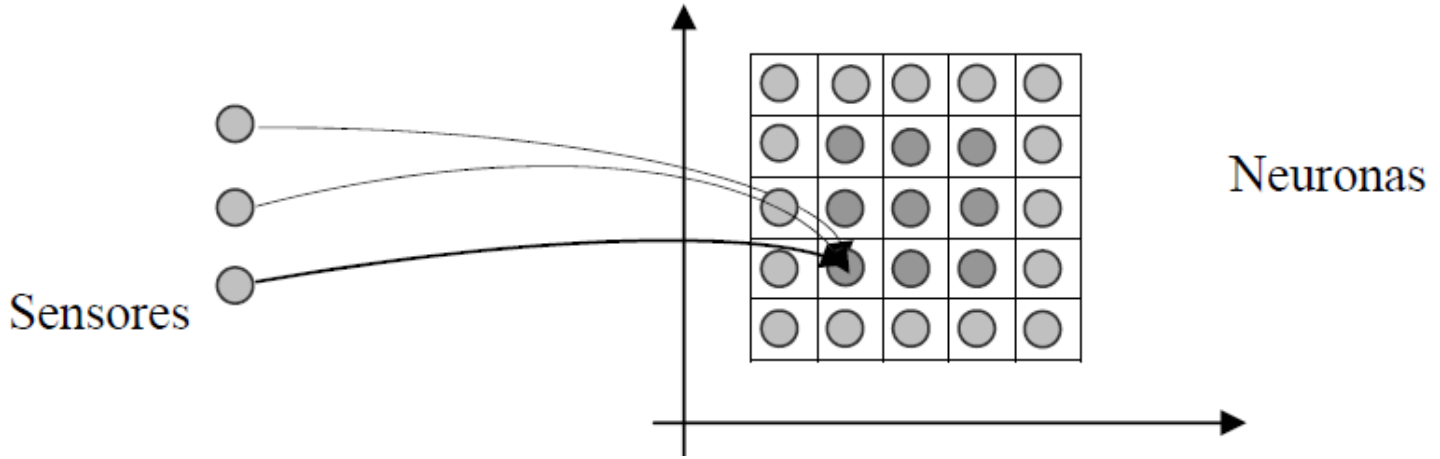


Autoorganización
Motivación biológica

- neuronas próximas físicamente presentan características y comportamiento similares
- las diferentes áreas de la corteza cerebral están caracterizadas por la delgadez de sus capas y por el tipo de neuronas que hay dentro de ellas; así tenemos las áreas visual, auditiva, motora, etc.
- cada entrada sensorial es aplicada al área correspondiente de la corteza cerebral de una manera ordenada
- la localización espacial de una neurona dentro de un mapa topográfico va a corresponder a un dominio o característica particular de los datos de entrada

Mapas Autoorganizados
Self-Organizing Maps (SOM)

- El objetivo de las redes autoorganizadas es descubrir patrones significativos o características en los datos de entrada mediante aprendizaje no supervisado.
- Sean $M1 \times M2$ unidades de proceso colocadas sobre una rejilla rectangular de manera que el vector $\vec{p}_i=(p_{i1},p_{i2})$ nos da la posición de la unidad de proceso i.



Mapas Autoorganizados
Self-Organizing Maps (SOM)

la proximidad entre las unidades de proceso i y j viene dada por

$$d(\vec{p}_i, \vec{p}_j) = \|\vec{p}_i - \vec{p}_j\|$$

d puede ser la **distancia Euclídea**

$$d(\vec{p}_i, \vec{p}_j) = \sqrt{(p_{i1} - p_{j1})^2 + (p_{i2} - p_{j2})^2}$$

la **distancia rectangular**

$$d(\vec{p}_i, \vec{p}_j) = |p_{i1} - p_{j1}| + |p_{i2} - p_{j2}|$$

o cualquier distancia

Mapas Autoorganizados
Self-Organizing Maps (SOM)

Se define una **función de vecindad** (o función de ventana) que tomará valores mayores conforme más próximas estén las dos unidades de proceso, es decir, es cualquier función decreciente de la distancia entre las mismas.

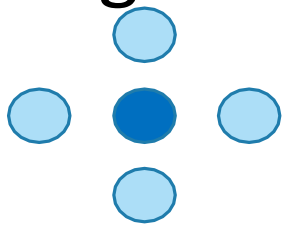
Ejemplo:

$$\Lambda(\vec{p}_i, \vec{p}_{i^*}) = \begin{cases} 1 & \text{si} \quad i = i^* \\ 0.5 & \text{si} \quad |i - i^*| = 1 \\ 0 & \text{en otro caso} \end{cases}$$

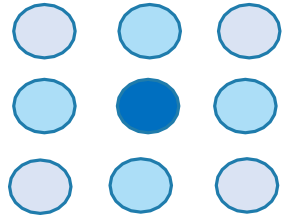
SOM (Ejemplos de función de vecindad)

- Si las unidades de proceso estuvieran sobre una cuadrícula se podría utilizar una función de vecindad determinada por una plantilla o ventana, como la siguiente

0	0.5	0
0.5	1	0.5
0	0.5	0



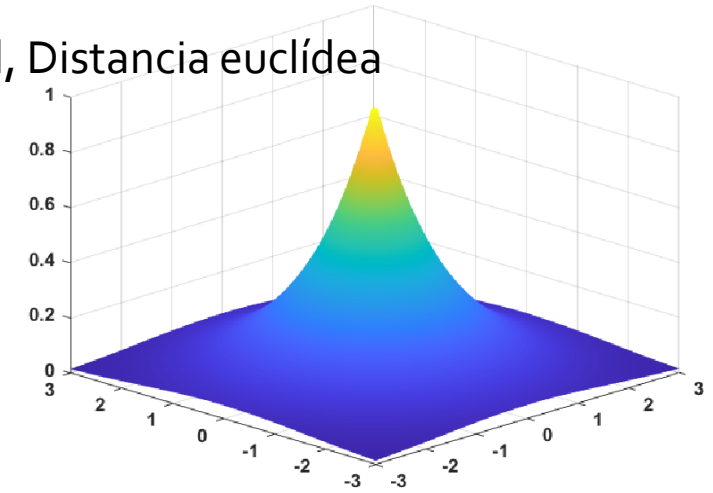
0.25	0.5	0.25
0.5	1	0.5
0.25	0.5	0.25



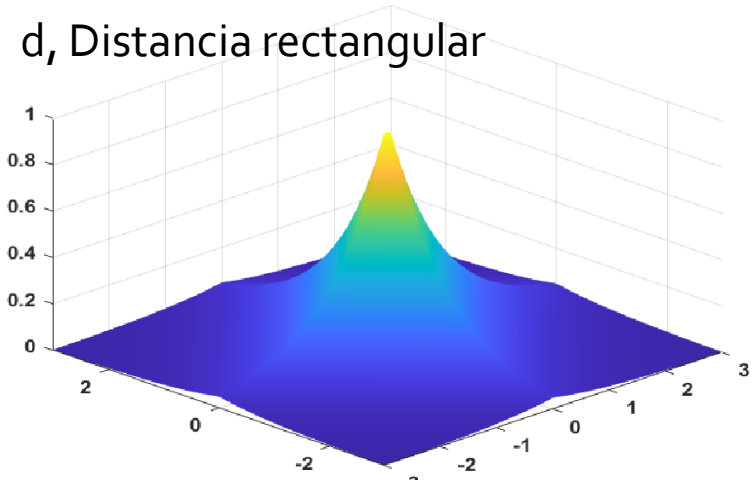
- cuando las unidades de proceso estuvieran ubicadas espacialmente se puede emplear como función de vecindad:

$$\Lambda(\vec{p}_i, \vec{p}_{i^*}) = e^{-d(\vec{p}_i, \vec{p}_{i^*})}$$

d, Distancia euclídea



d, Distancia rectangular



SOM (Ejemplos de función de vecindad)

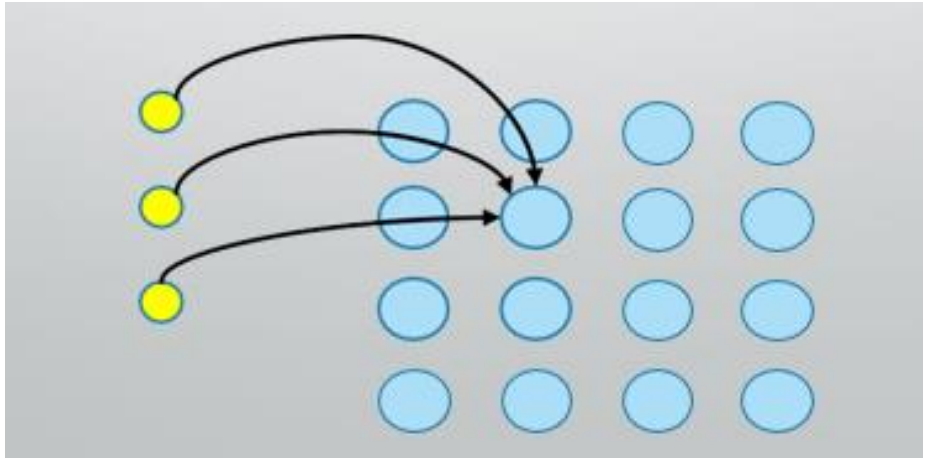
- se puede utilizar funciones de vecindad dinámicas, es decir, que se van modificando en cada iteración. Por ejemplo

$$\Lambda(\vec{p}_i, \vec{p}_{i^*}) = e^{-\frac{d(\vec{p}_i, \vec{p}_{i^*})}{\sigma(k)^2}}$$

donde $\sigma(k)^2 = \sigma_0 \, (1 + k)^{-1/2}$, va reduciendo su radio de vecindad conforme *k* aumenta aproximándose sucesivamente a un aprendizaje competitivo simple.

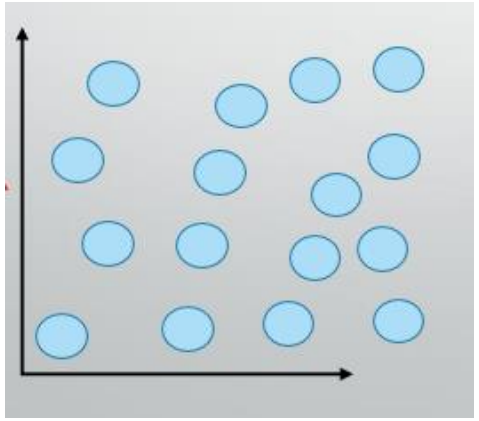
SOM (Arquitectura)

- La arquitectura es la misma de una red competitiva, es decir, una única capa donde cada neurona tendrá tantos sensores de entradas como dimensión tengan los patrones.
- No existen pesos sinápticos entre diferentes neuronas.
- La topología y la función de vecindad determinarán cómo se relacionan entre sí las diferentes neuronas de la red.

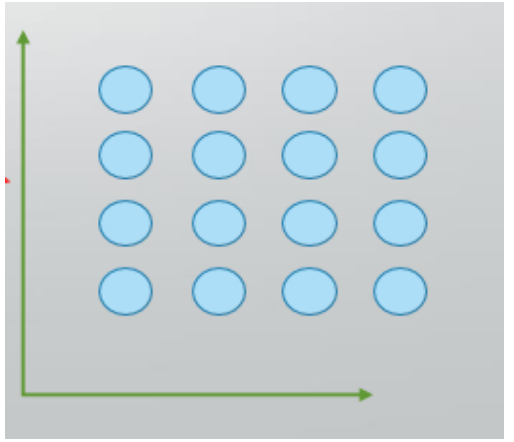


SOM (Topología)

- La posición en el espacio de entrada de una neurona está determinada por sus pesos sinápticos, al igual que en la red competitiva, varía conforme la red aprende.
- La posición de cada neurona en el espacio topológico determina la **topología** de la red. **No evoluciona**, es constante a lo largo del tiempo.



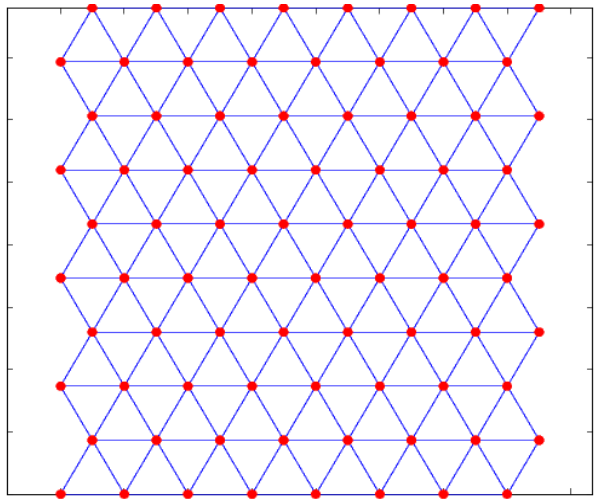
Espacio de entrada

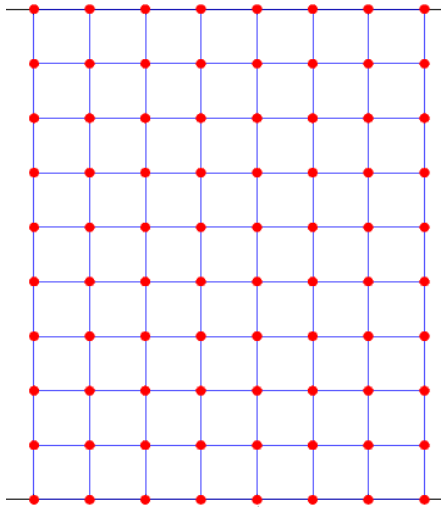


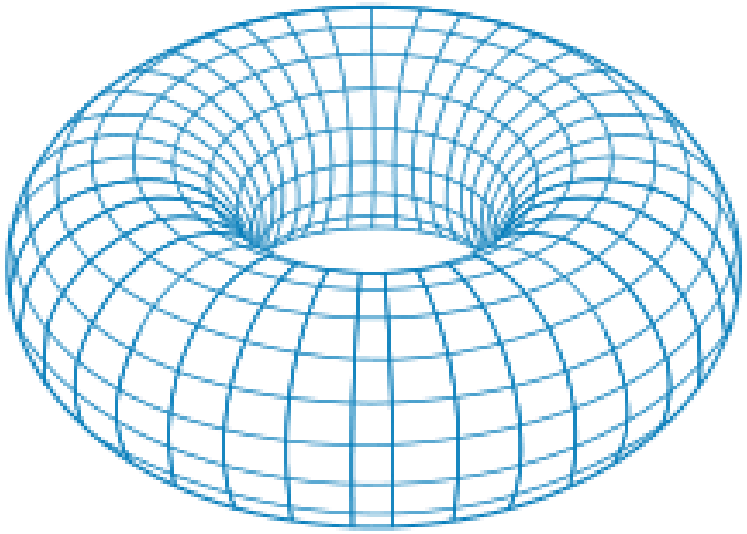
Espacio topológico

SOM (Topología)

Normalmente una SOM está dispuesta siguiendo una topología de malla o rejilla. La topología, por medio de la función de vecindad, determina el grado de vecindad entre neuronas, pero NO sus conexiones sinápticas.

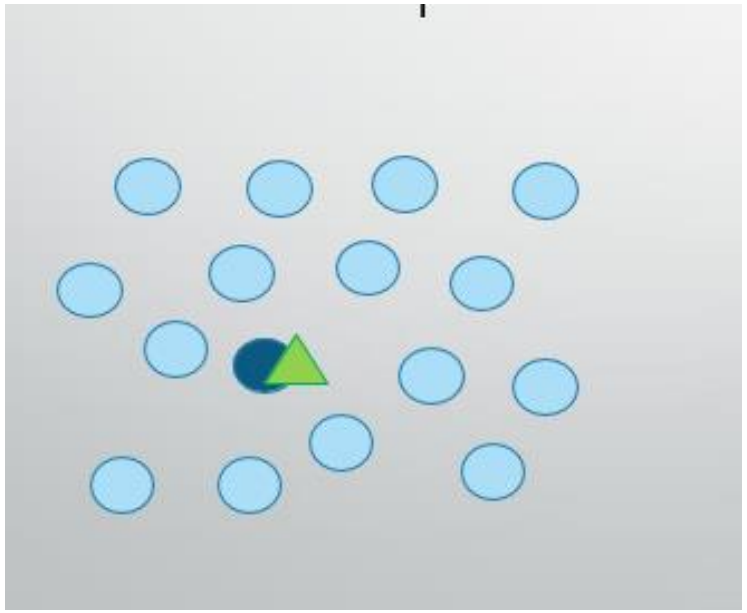






SOM (Dinámica de la Computación)

La dinámica de la computación es análoga a la de la red competitiva. Existe una única neurona ganadora para cada patrón de entrada, que es la que más cerca está de dicho patrón



- Potencial sináptico:

$$h_i = w_{i1}x_1 + w_{i2}x_2 + \ldots + w_{iN}x_N - \theta_i$$

$$\theta_i = \frac{1}{2}\left(w_{i1}^2 + w_{i2}^2 + \cdots + w_{iN}^2\right)$$

$$Y_i = \begin{cases} 1 \text{ si } & h_i = \max\{h_1, h_2, \ldots, h_P\} \\ 0 & \text{en otro caso} \end{cases}$$

i=1,2,…,P

SOM (Aprendizaje)

La **posición de las neuronas** en el espacio de entrada evoluciona conforme aprende la red.

La neurona ganadora i^* se adapta al patrón de entrada de forma similar a la red competitiva, pero en este caso, los vecinos de la ganadora también se adaptan, pero en menor medida.

$$\vec{w}_i(k+1) = \vec{w}_i(k) + \mu(k)\Lambda(\vec{p}_{i^*}, \vec{p}_i)[\vec{x}(k) - \vec{w}_i(k)], \; 1=1,2,..,M$$

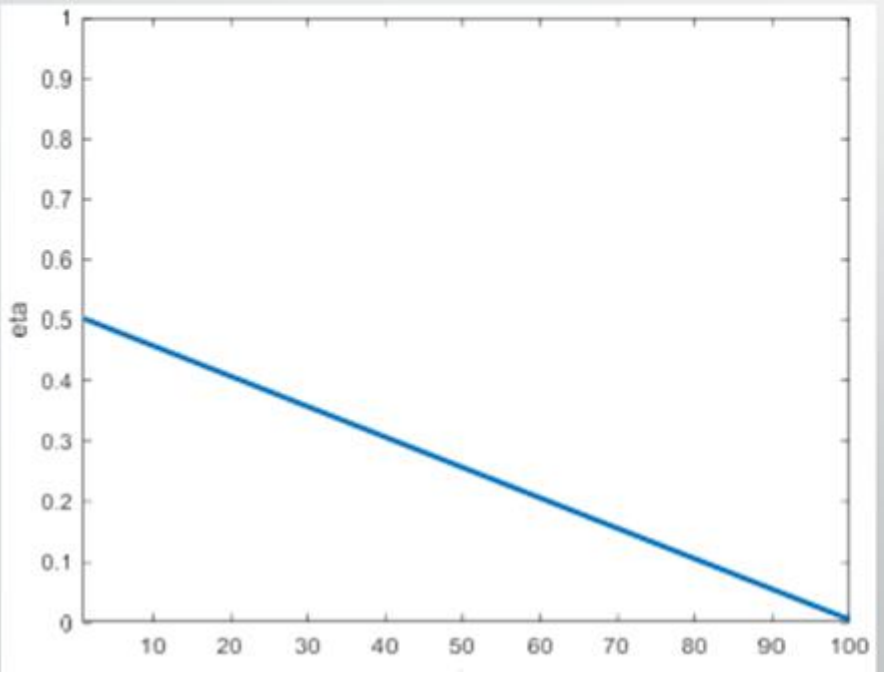
$$\mu(k)=\mu_0\left(1-\frac{k}{T}\right) \qquad \mu(k)=\mu_0(1+k)^{-1/2}$$

SOM (Aprendizaje)

$$\vec{w}_i(k+1) = \vec{w}_i(k) + \mu(k)\Lambda(\vec{p}_{i^*}, \vec{p}_i)[\vec{x}(k) - \vec{w}_i(k)],\ 1=1,2,..,M$$

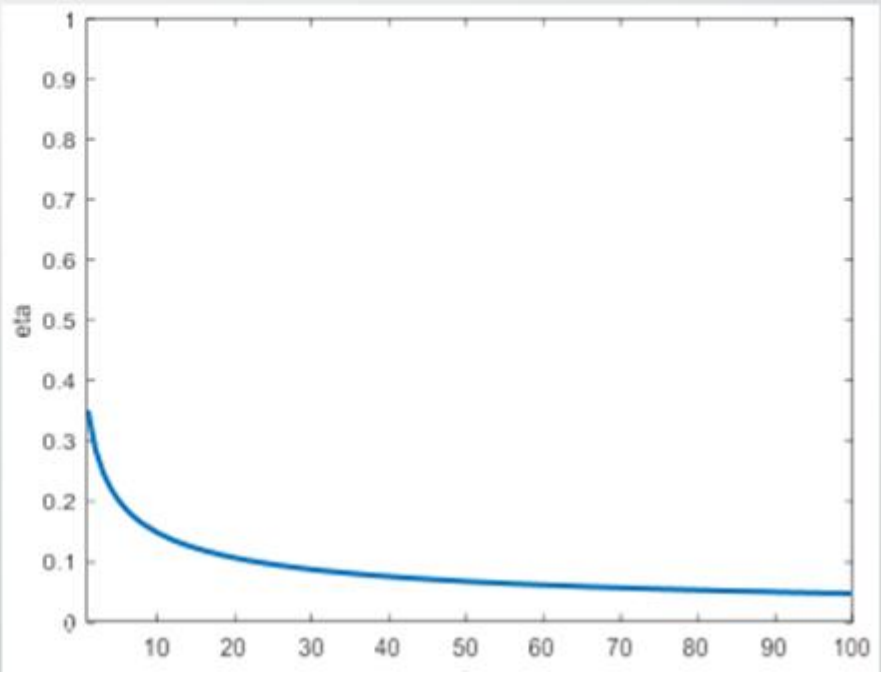
$$\eta(k) = \eta_0\left(1 - \frac{k}{T}\right) \qquad \eta_0 = 0.5$$

(caída lineal)



$$\eta(k) = \eta_0(1+k)^{-1/2} \qquad \eta_0 = 0.5$$

(caída no lineal)



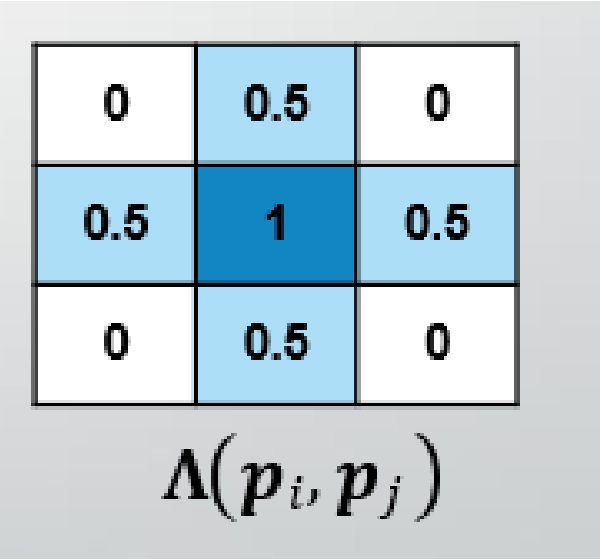
SOM (Aprendizaje): Ejercicio

- Tenemos una SOM con 4 neuronas cuya topología es una rejilla 2x2
- Los valores actuales para los pesos sinápticos de las cuatro neuronas son los siguientes

$$\vec{w}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \vec{w}_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \vec{w}_3 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \vec{w}_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

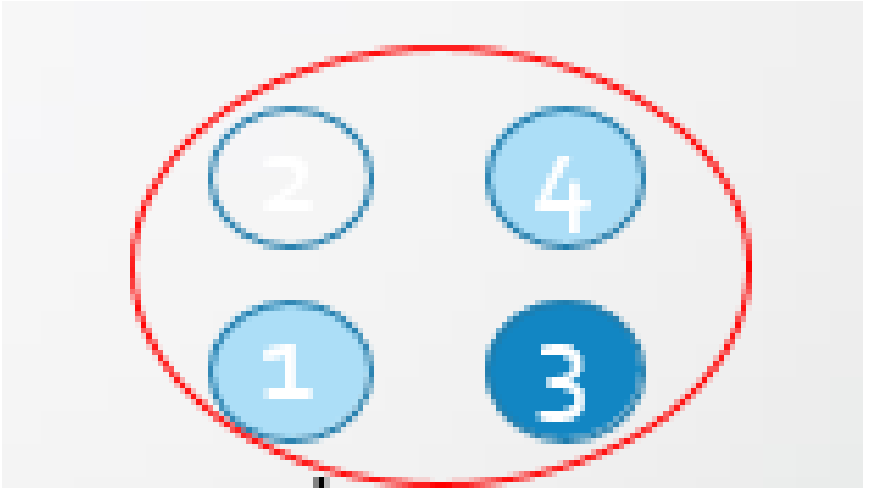


- Actualizar los pesos para la etapa $k + 1$, cuando se presenta el patrón de entrada $\vec{X}(k) = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ utilizando $\eta(k) = 0.5$



SOM (Aprendizaje): Ejercicio

$$h_1(k) = 4 * 0 + 2 * 1 - 0.5(0 + 1) = 1.5$$
$$h_2(k) = 4 * 1 + 2 * 2 - 0.5(1 + 4) = 5.5$$
$$h_3(k) = 4 * 2 + 2 * 1 - 0.5(4 + 1) = 7.5$$
$$h_4(k) = 4 * 2 + 2 * 4 - 0.5(4 + 16) = 6$$



$$w_{31}(k + 1) = 2 + 0.5(4 - 2) = 3$$
$$w_{32}(k + 1) = 1 + 0.5(2 - 1) = 1.5$$

$$w_{11}(k + 1) = 0 + 0.25(4 - 0) = 1$$
$$w_{12}(k + 1) = 1 + 0.25(2 - 1) = 1.25$$

$$w_{41}(k + 1) = 2 + 0.25(4 - 2) = 2.5$$
$$w_{42}(k + 1) = 4 + 0.25(2 - 4) = 3.5$$

SOM (Aprendizaje): Ejercicio

- Los valores actuales de los pesos son

$$\vec{w}_1(k) = \begin{pmatrix}0\\1\end{pmatrix}, \vec{w}_2(k) = \begin{pmatrix}1\\2\end{pmatrix},$$

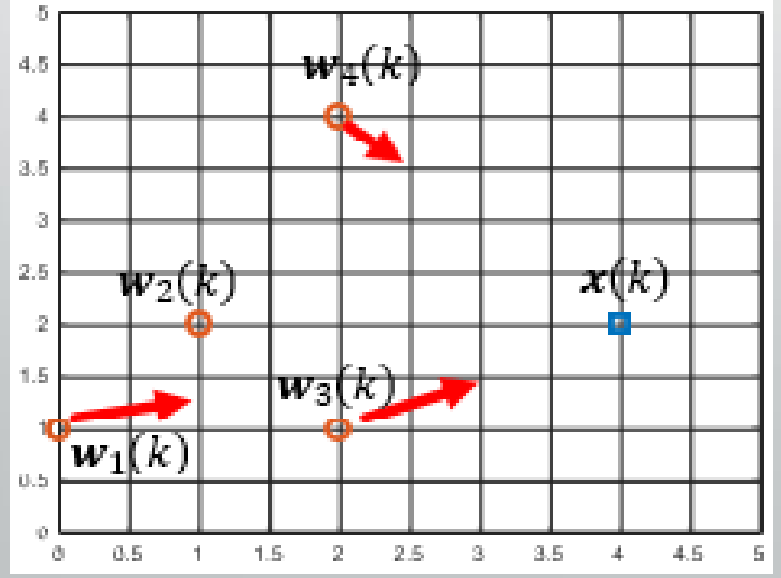
$$\vec{w}_3(k) = \begin{pmatrix}2\\1\end{pmatrix}, \vec{w}_4(k) = \begin{pmatrix}2\\4\end{pmatrix}$$

- Los valores nuevos de los pesos son

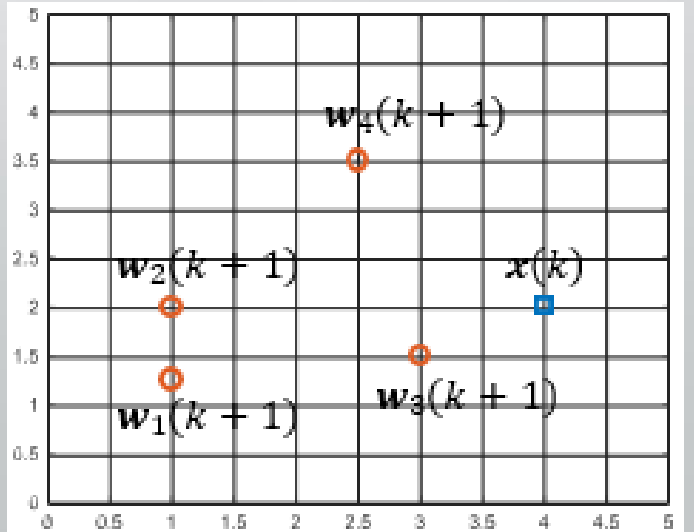
$$\vec{w}_1(k+1) = \begin{pmatrix}1\\1{,}25\end{pmatrix}, \vec{w}_2(k+1) = \begin{pmatrix}1\\2\end{pmatrix},$$

$$\vec{w}_3(k+1) = \begin{pmatrix}3\\1{,}5\end{pmatrix}, \vec{w}_4(k+1) = \begin{pmatrix}2{,}5\\3{,}5\end{pmatrix}$$

k

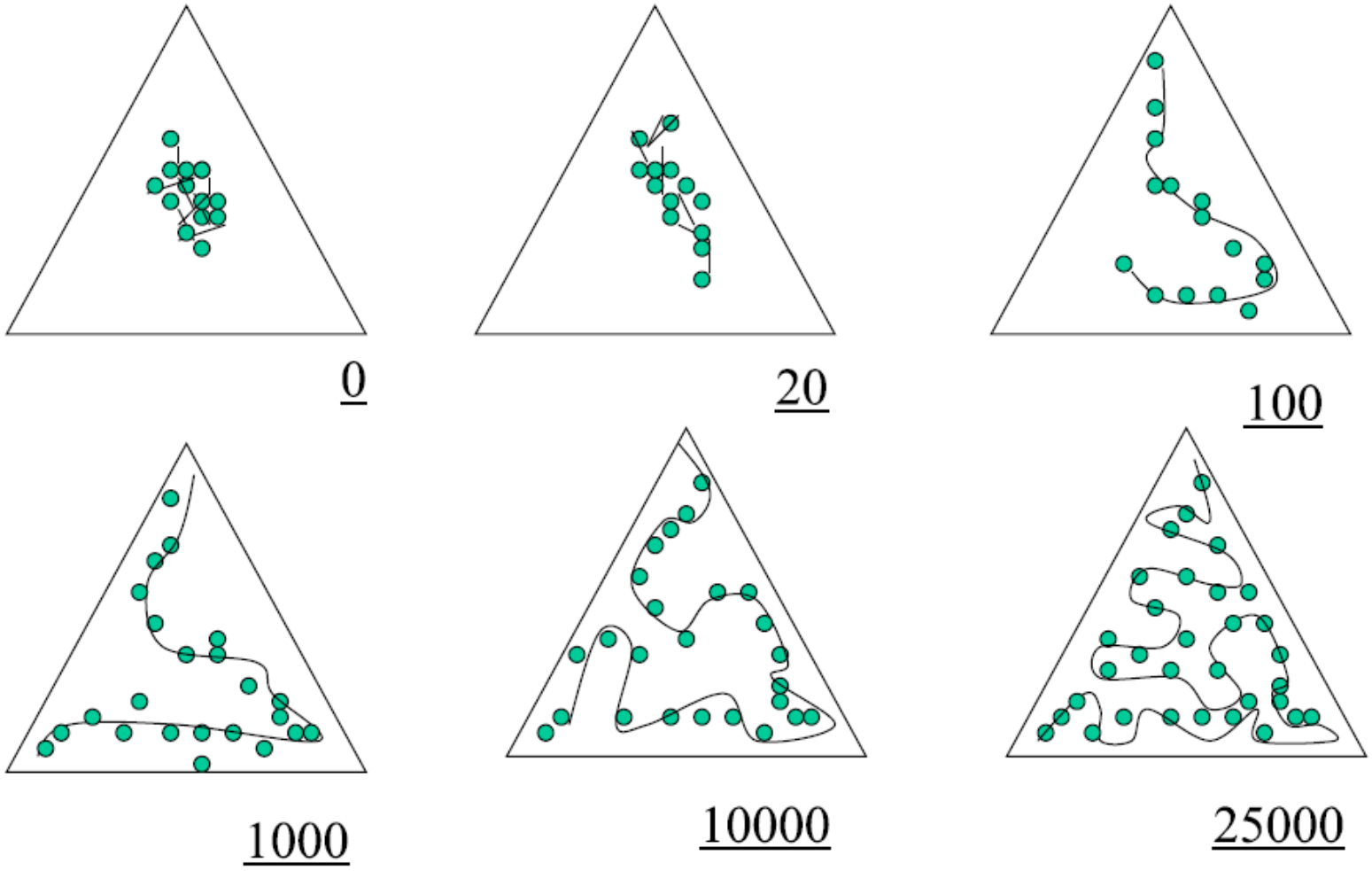


k+1



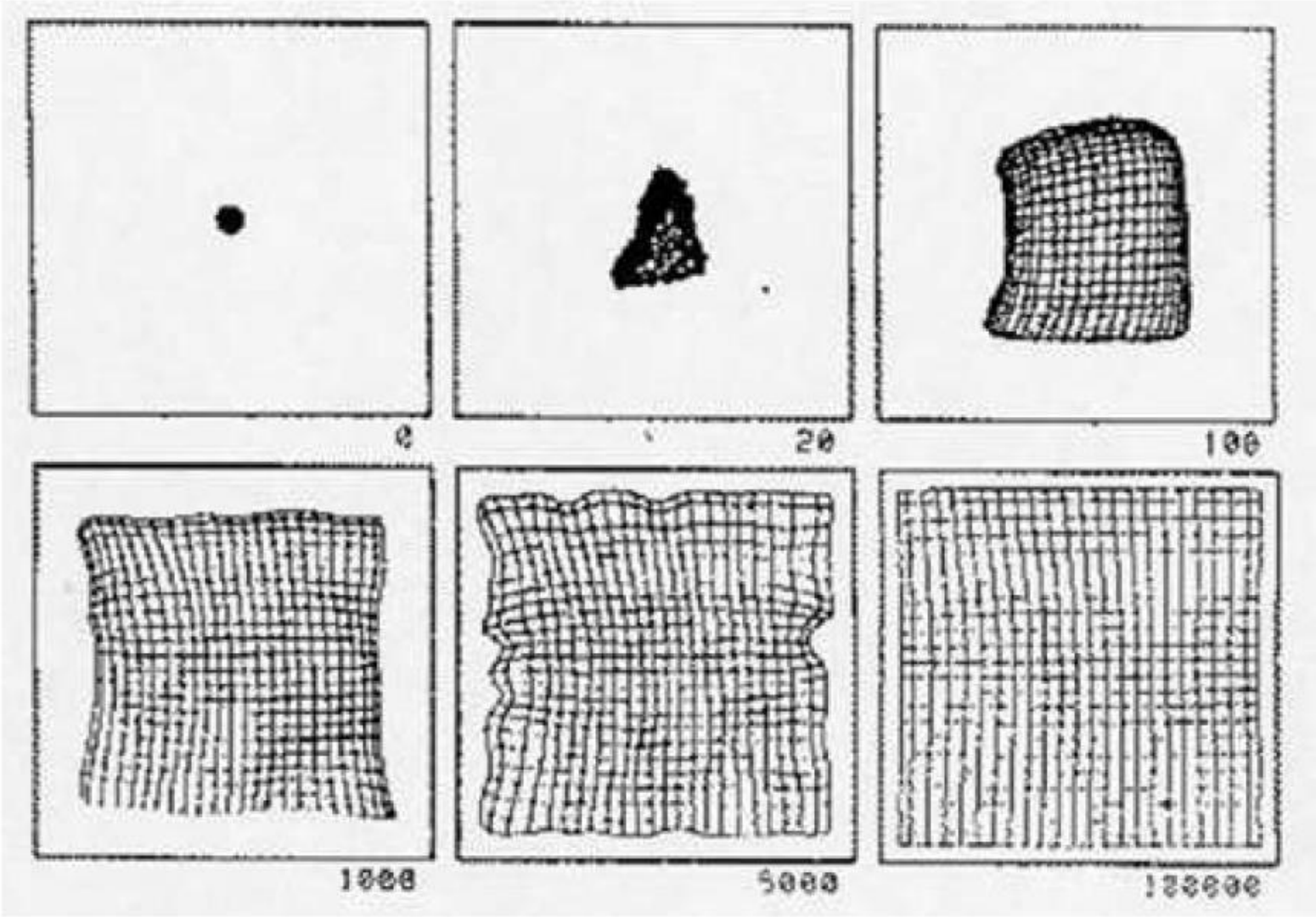
SOM (Aprendizaje): Ejercicio

Aprendizaje usando puntos de un espacio bidimensional (triangular)



SOM (Aprendizaje): Ejercicio

Aprendizaje usando puntos de un espacio bidimensional (cuadrado)



Ejercicio

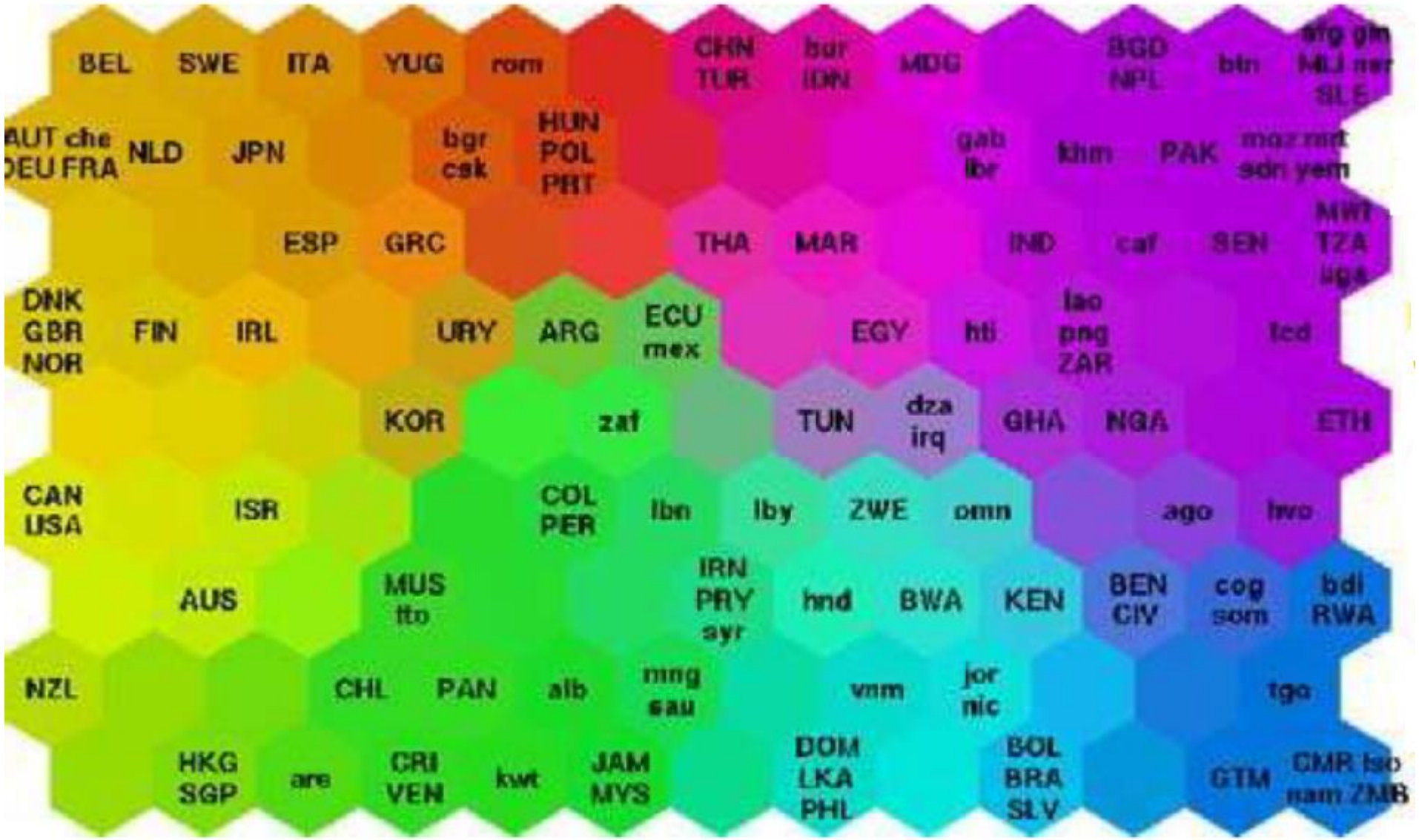
https://www.youtube.com/watch?v=B2Z-IFHcMxg

SOM (Aprendizaje): Ejercicio

Nivel de pobreza

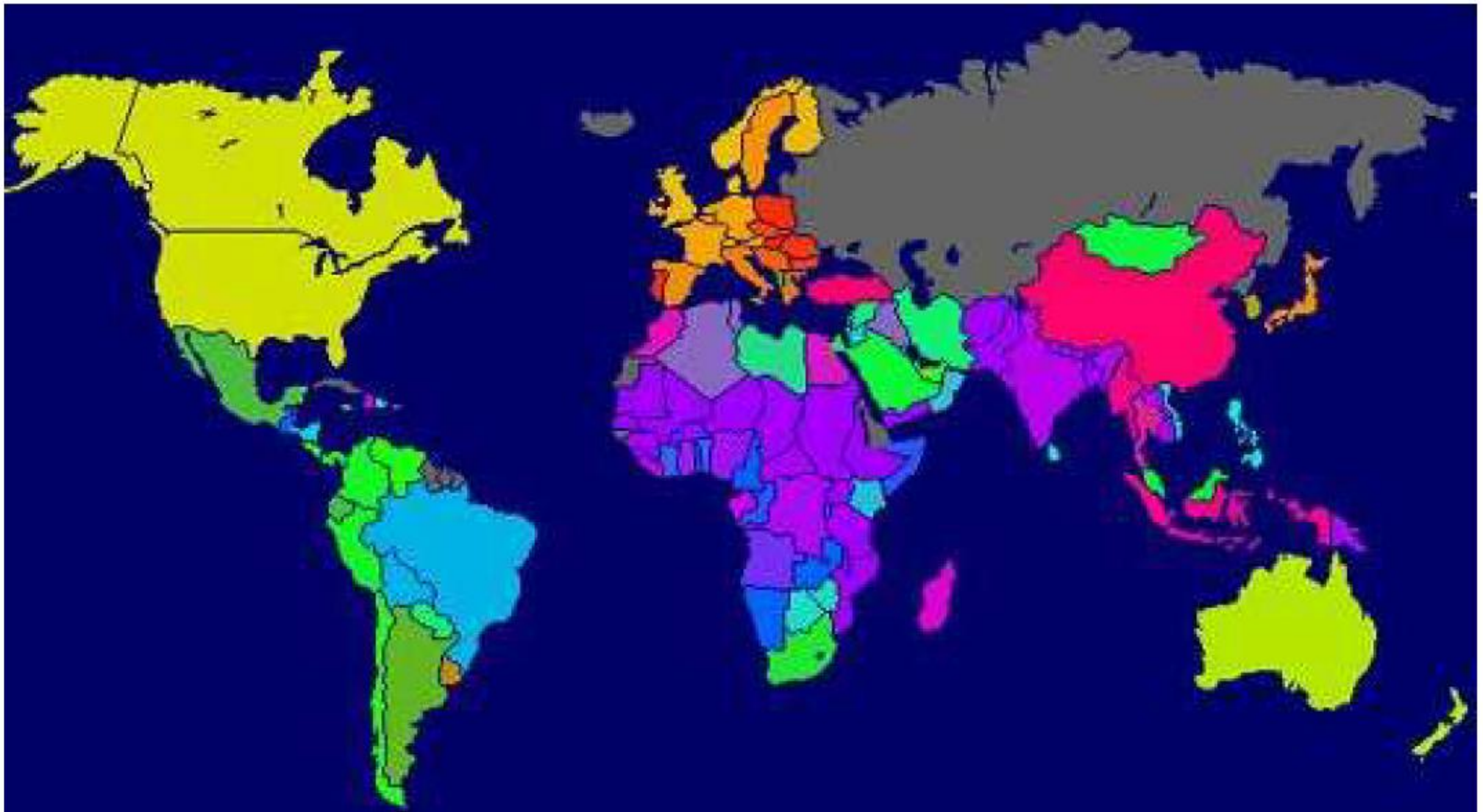
Se usar SOM una para representar las agrupaciones que forman los diversos países en una red 2D. Se han usado datos de entrada que corresponden a las características socio-económicas de cada país (tipo de sistema de salud, PIB, indicadores educativos,…).

Los países con unos factores de calidad de vida similares terminaran en el mismo grupo.



Cada hexágono es una neurona de la SOM.
Arriba a al izquierda están los países con mejor calidad de vida y abajo a la derecha los más afectados por la pobrezaa

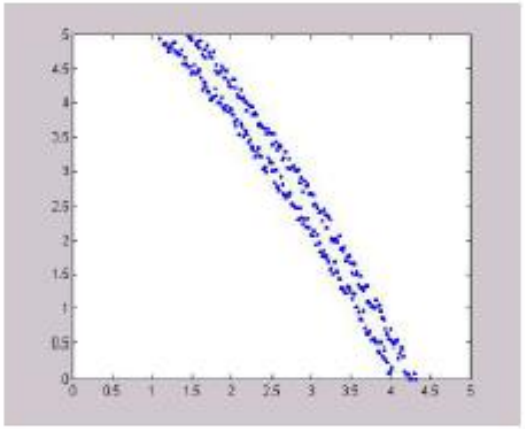
Una vez extraídos los colores, podemos volver a proyectar los países en un mapa estándar:



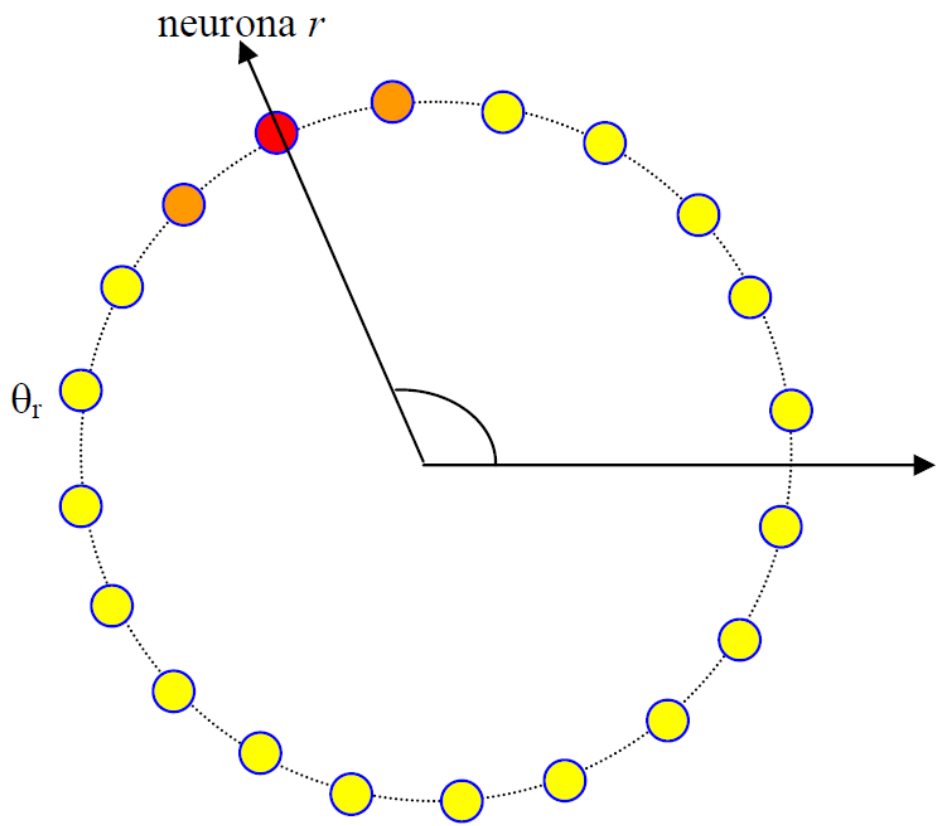
SOM (Aprendizaje): Ejercicio

Se dispone de 322 puntos (xi, yi), i=1,2,...,322, que configuran el contorno difuso de un vaso sanguíneo en una mamografía. Se trata de diseñar una red neuronal que construya el contorno poligonal de 20 vértices que “mejor” se ajusta al contorno del vaso sanguíneo.





~~SOM (Aprendizaje): Ejercicio~~



Topología de la red autoorganizada

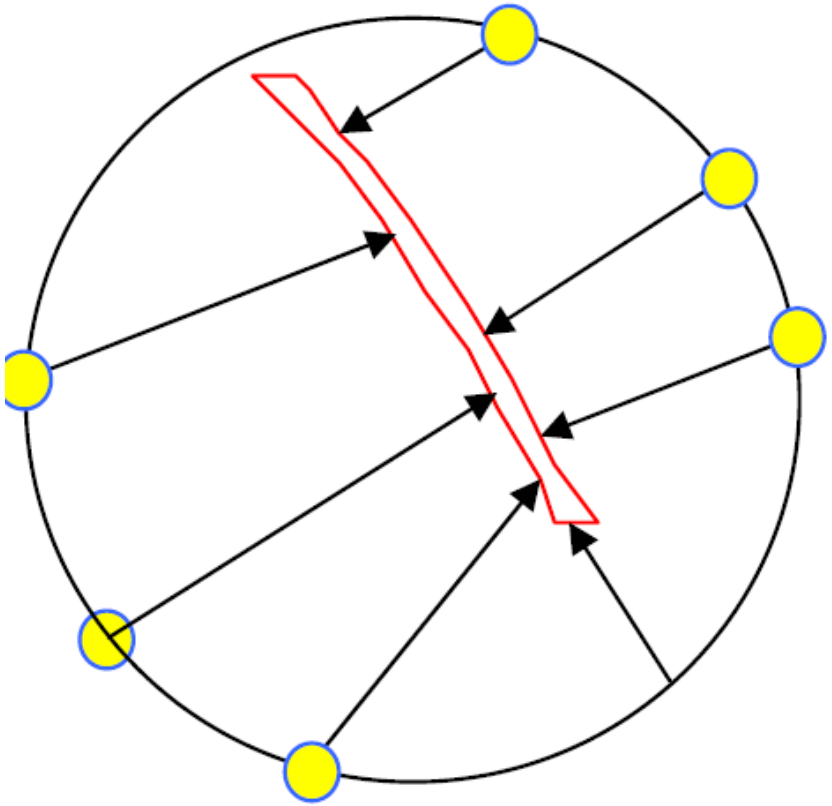
$$\Lambda(\theta_r,\theta_j)=\begin{cases}1 & \text{si } |\theta_r-\theta_j|=0\\ 1/2 & \text{si } |\theta_r-\theta_j|=\dfrac{\pi}{10}\\ 0 & \text{en otro caso}\end{cases} \quad , i=1,2,\ldots,N$$

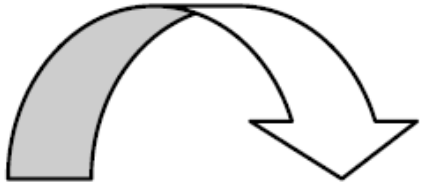
Función de vecindad

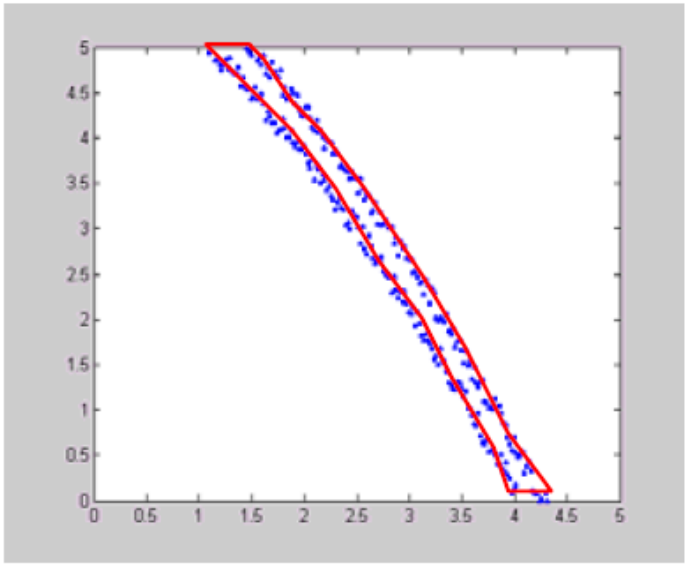
***r** es* la neurona ganadora

θ*i* el ángulo, en radianes, que determina la posición de la neurona *i* con respecto al origen y el eje de abscisas

SOM (Aprendizaje): Ejercicio







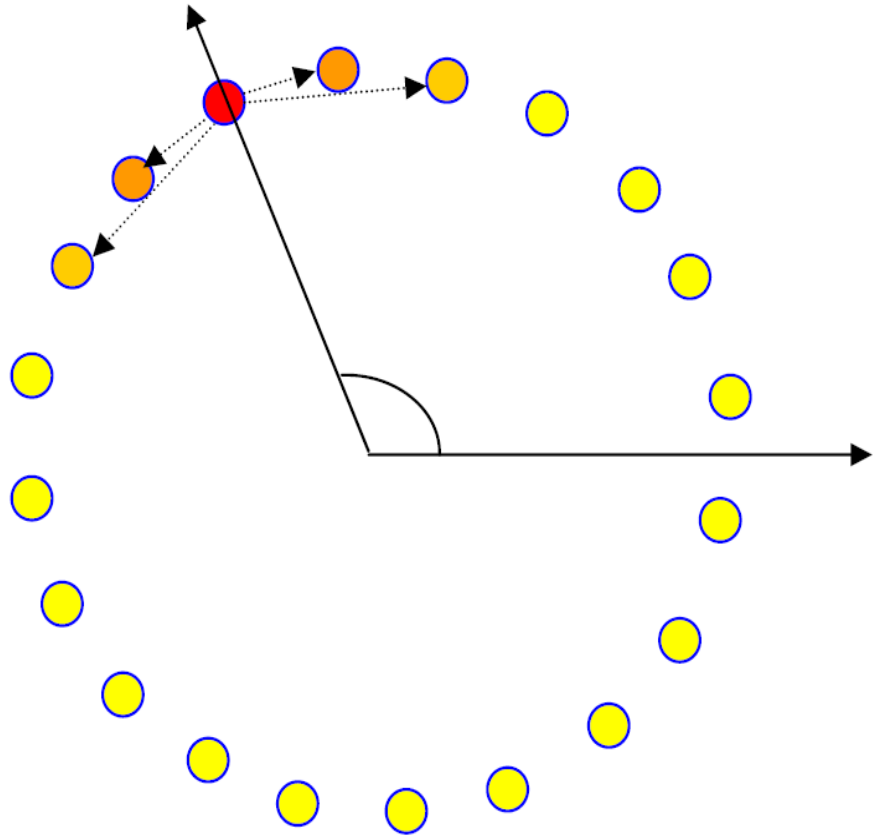
Ajuste al contorno de vaso sanguíneo de una mamografía

SOM (Aprendizaje): Ejercicio

Diseña una red SOM que resuelva el problema del viajante de comercio (Traveling Salesman Problem, TSP).

- Hay N ciudades a recorrer
- La solución del problema del viajante es un recorrido (ruta) que pasa por cada una de las N ciudades sólo una vez y regresa a la ciudad de partida con menor distancia
- La red autoorganizada tendrá más unidades de proceso que ciudades (por ejemplo, 3*N*)

SOM (Aprendizaje): Ejercicio

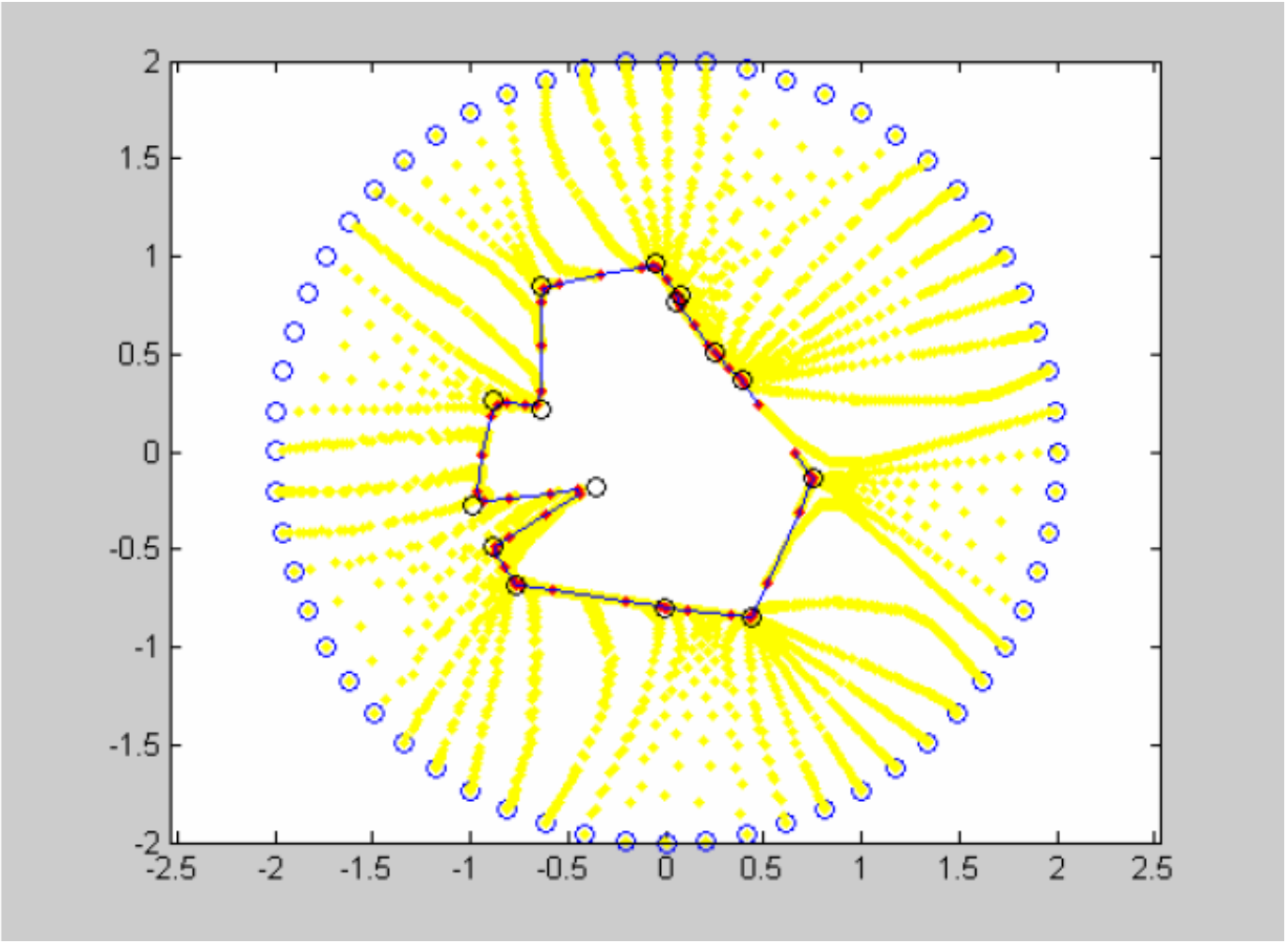


Topología de la red autoorganizada

$$\Lambda(\theta_r,\theta_j)=\begin{cases}1 & \text{si } |\theta_r-\theta_j|=0\\ 1/2 & \text{si } |\theta_r-\theta_j|=\dfrac{2\pi}{3N}\\ 1/4 & \text{si } |\theta_r-\theta_j|=\dfrac{4\pi}{3N}\\ 0 & \text{en otro caso}\end{cases}, \quad i=1,2,\ldots,3N$$

Función de vecindad: ***r*** la neurona ganadora y **θ*i*** el ángulo, en radianes, que determina la posición de la neurona *i* con respecto al origen y el eje de abscisas

SOM (Aprendizaje): Ejercicio



Evolución de los pesos sinápticos:

Los vectores sinápticos serán atraídos por los puntos donde están situadas las ciudades y cuando se estabilice la red los pesos sinápticos serán las coordenadas de las ciudades

La ruta viene determinada por la secuencia de ciudades sobre la circunferencia.

Conjunto de entrenamiento de N puntos: $\{(xi,yi),\ i=1,2,...,N\}$ que corresponden a las coordenadas de la ciudades

Redes competitivas (Aprendizaje supervisado)

- Tenemos un conjunto de patrones para los que conocemos la clase C_S a la que realmente pertenecen
- Regla de aprendizaje. Los pesos se modifican de la siguiente manera:
  - ✓Si acierta, se sigue el sentido opuesto al gradiente (igual que el aprendizaje no supervisado).
  - ✓Si falla, se sigue el mismo sentido del gradiente.

Redes competitivas (Aprendizaje supervisado)

- Si en la iteración k, r es la neurona ganadora, la actualización de pesos es:

$$\vec{w}_r(k+1) = \begin{cases} \vec{w}_r(k) + \eta_r(k)\left(\vec{x} - \vec{w}_r(k)\right) si\ r = s \\ \vec{w}_r(k) - \eta_r(k)\left(\vec{x} - \vec{w}_r(k)\right) si\ r \neq s \end{cases}$$

- $\Delta\ \vec{w}_i(k+1)$=0 si $i \neq r$

las neuronas no ganadoras no modifican sus pesos

$$\eta_r(k+1) = \begin{cases} \dfrac{\eta_r(k)}{\eta_r(k)+1}, si\ r = s \\ \dfrac{\eta_r(k)}{\eta_r(k)-1}, si\ r \neq s \end{cases}$$

Reduce lo "brusco" que son los cambios en el vector de pesos conforme va acertando, se estabiliza

Si ha habido un fallo, "parece que algo no va bien", "se prepara" para modificar más el vector de pesos

Redes competitivas (Aprendizaje supervisado)

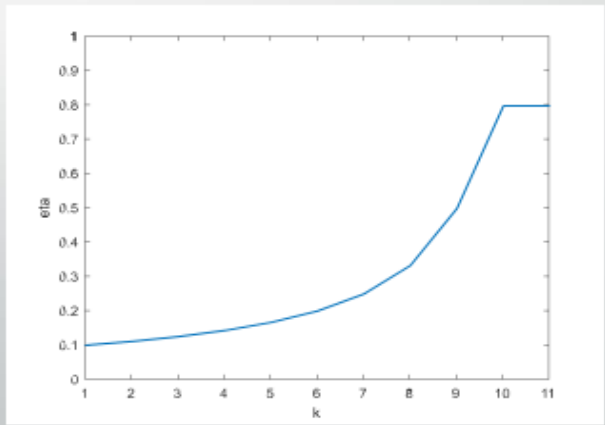
- La modificación de la tasa de aprendizaje siguiendo la ecuación anterior puede llevar a valores absurdos.

Imaginemos que se equivoca varias veces consecutivas partiendo de una tasa inicial de 0.1:

$0.1 \rightarrow 0.111 \rightarrow 0.125 \rightarrow 0.143 \rightarrow 0.167 \rightarrow 0.200 \rightarrow 0.250 \rightarrow 0.333 \rightarrow 0.50 \rightarrow 1 \rightarrow \infty$

- En general es mejor utilizar la siguiente regla de modificación de la tasa de aprendizaje:

$$\eta_r(k+1) = \begin{cases} \dfrac{\eta_r(k)}{\eta_r(k)+1}, si\ r = s \\ \min\left\{\dfrac{\eta_r(k)}{\eta_r(k)-1}, \eta_r^{max}\right\}, si\ r \neq s \end{cases}$$



Ejemplo de $\eta_r\ (1) = 0.1, \eta_r^{max} = 0.8$ y 10 fallos seguidos

Redes competitivas (Aprendizaje supervisado)

- Interpretación gráfica de la regla de aprendizaje

